SQL Interview Q_A

1. **What is Database?**

   A collection of data in the form of tables. The database provides the capability to access and manipulate data.

2. **What is DBMS?**

   DBMS stands for **Database Management System**. DBMS is a system software responsible for the creation, retrieval, updation, and management of the database.

3. **Types of databases?**

   1. **Relational database Management System (RDBMS):** it will have a relation between rows and columns.
      Ex: MySQL, SQL Server, Postresql,sqlite,MariaDB,oracle, IBM DB2

   2. **No-Sql database:** key value, document, graph.
      Ex: Hbase,MangoDB,Cassandra

4. **What is the difference between SQL and MySQL?**

   SQL (Structured Query Language) is for managing and manipulating relational databases.
   It allows users to do CRUD operations **(create, read, update, and delete data).**

   MySQL is a **Relational database management system**, like SQL Server, Oracle,IBM DB2 etc.

5. **What are types of SQL Commands?**

   1.DDL- data definition language – Used to define and manage database structure (data base and tables).
   - **CREATE**
   - **DROP**
   - **ALTER**
   - **TRUNCATE**

   2.DML- Used to manipulate data within tables.
   - **SELECT**
   - **INSERT**
   - **UPDATE**

- **DELETE**
- **MERGE**

3. DCL (Data Control Language): Deals with permissions and access control
   - **GRANT**
   - **REVOKE**

4. TCL (Transaction Control Language): Manages transactions
   - **COMMIT**
   - **ROLLBACK**
   - **SAVEPOINT**

6. Data Types?

1. String

1. **CHAR(n) –** Fixed length text.
   Always stores n characters (pads with spaces).
   Max Length: 0–255
   Example:
   CHAR(5) → 'ABC '
   Use Case: Fixed codes (e.g., country codes).

2. **VARCHAR(n) –** Variable length text.
   Stores only what you enter.
   Max Length: Up to 65535*
   Example:
   VARCHAR(20) → 'ABC'
   Use Case: Names, addresses.

3. **TEXT –** Large text.
   Max Length: Up to 2GB
   Example:
   'THIS IS A LONG NOTE'
   Use Case: Long descriptions (avoid indexing).

2. Numeric

1. **INT –** Whole number.
   Range: -2,147,483,648 to 2,147,483,647
   Example: 25

Use Case: IDs, counters.

2. **BIGINT –** Very big number.
   Example: 9876543210
   Use Case: Large datasets.

3. **FLOAT –** Decimal (approx).
   Example: 5.87
   Use Case: Scientific data (may have rounding errors).

4. **DECIMAL(p,s) –** Decimal (exact).
   p = total digits, s = digits after decimal.
   Example: DECIMAL(5,2) → 123.45
   Use Case: Financial data**.**

3. Date and Time

1. **DATE** – Date only.
   Format: YYYY-MM-DD
   Example: 2025-12-31

2. **TIME** – Time only.
   Format: HH:MM:SS
   Example: 16:30:00

3. **DATETIME –** Date + Time.
   Format: YYYY-MM-DD HH:MM:SS
   Example: 2025-12-31 16:30:00

7. CRUD operations?

CREATE – creating records using insert statements.
READ- reading records using select statements.
UPDATE- updating records using update statements.
DELETE- deleting records using deleted statements.

8. What are Tables, rows/records, and Columns/ Fields/Attributes?

- A **table** is a physical table.
- It consumes physical space in a database to store data.
- **Table** is independent.

- Tables hold the data in the form of rows and columns like excel spreadsheet.
- **Columns/ Fields/Attributes** can be categorized as **vertical.**
- **rows/records** as **horizontal**.

9. What are Temporary Table?

A Temporary Table is also a physical table but used to store data temporarily. Temporary tables are typically used to store intermediate results or to break down complex queries into simpler ones. They are often created and used within a single session and automatically dropped when the session ends.

10. What is a View?

- **View** is a virtual table based on queries.
- **View** does not store data in the database, it will store only the query expression.
- When view query executed it dynamically retrieves data based on a pre-defined "SELECT query" statement.
- It can fetch data from one or more tables.
- **View** is dependent on one or more tables

11. Materialized Views in SQL?

- Materialized views are also virtual table, but the result of the query is stored physically in the database.
- It can be accessed faster, and the performance of the materialized views is better than normal views. This is because the data is stored in the database.
- Sometimes, materialized views are also called "indexed views"
- Materialized Views are used when data is to be accessed frequently and data in table does not get updated on frequent basis.
- If the Data in tables are updated, we need to refresh Materialized Views.
- It consumes more space in database than normal view.

Creating a Materialized View:

```
CREATE MATERIALIZED VIEW mv_sales_summary

AS

SELECT customer_id, SUM(sales_amount) AS total_sales

FROM sales

GROUP BY customer_id;
```

Refreshing the Materialized View:

REFRESH MATERIALIZED VIEW mv_sales_summary;

12. What are Constraints in SQL?

SQL constraints are rules applied to **columns to** limit the type of data that can be **inserted**, **updated**, or **deleted**. These rules ensure the data is valid, consistent, and as per the business logic.

1. **NOT NULL** - constraint ensures that a column cannot contain NULL values.
2. **CHECK** - constraint allows us to specify a condition that data must satisfy before it is inserted into the table..
3. **DEFAULT** - constraint provides a default value for a column when no value is specified during insertion.
4. **UNIQUE KEY**
5. **PRIMARY KEY**
6. **FOREIGN KEY**

13. What is a Primary Key?

- It uniquely identifies each row in a table.
- For primary key null and repeated values not allowed. This constraint is a combination of the NOT NULL and UNIQUE constraints.
- A table can have only one primary key.
- **Composite primary key** can have more than one or more column/field.
- Auto increment key is best for primary key.

14. What is a UNIQUE Key?

- constraint ensures that all values in a column are distinct across all rows in a table.
- We can have multiple unique keys in a table.
- Unique key can hold null.
- My sql db can hold any number of nulls, some db's can hold only one null.

15. Primary key Vs Unique key?

we can have only one Primary key and multiple unique keys in a table.

16. What is a Foreign Key?

- constraint links a column in one table to the primary key in another table.
- The foreign key constraint is used to prevent actions that would destroy links between two tables.
- The table with foreign key is called child table.
- The table with primary key is called parent/reference table.

17. What is the SELECT statement?

The SELECT statement is the most used command in Structured Query Language. It is used to access the records from one or more database tables and views.

```
SELECT * FROM myDB.students;
```

18. What are some common clauses used with SELECT query in SQL?

The sql clauses can help filter out the data according to the users' needs. The main clauses are:
SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, INSERT, UPDATE, DELETE, and JOIN

19. What is a Query?

A query is a request for data/information from a table in a database. A database query can be either a select query or an action query.

```
/* select query */

SELECT FNAME, LNAME
FROM MYDB.STUDENTS
WHERE STUDENT_ID = 1;

/* action query */

UPDATE MYDB.STUDENTS
SET FNAME = 'CAPTAIN', LNAME = 'AMERICA'
WHERE STUDENT_ID = 1;
```

20. What is the difference between Drop, DELETE and TRUNCATE statements?

Drop:

- It's DDL command.
- DROP command is used to remove an object from the database. If you drop a table, all the rows in the table are deleted and the table structure is removed from the database.

  EX: DROP TABLE Candidates;

Delete:

- It's DML command.
- It's used to delete individual records and we can also delete all records.
- It's Drops the records one after another, so it takes more time than truncate, to drop all table records.

  EX: DELETE FROM Candidates WHERE CandidateId > 1000;

Truncate:

- It's DDL command.
- It also removes all records.
- truncate internally drop the table and recreates it.
- Truncate is more efficient than delete.

  EX: TRUNCATE TABLE Candidates;

21. What is a Join? List its different types.

is used to combine records (rows) from two or more tables in a SQL database based on a related column between the two.

There are four different types of JOINs in SQL:

(INNER) JOIN:
Only matching records from both left and right tables are considered. Nonmatching records are discarded.

Syntax:

```
SELECT *  FROM table1
INNER JOIN table2
ON table1.Column_Name = table2.Column_Name;
```

LEFT (OUTER) JOIN:
All matching records from the left and right table are considered.
And all non-matching records in the left table which do not have match in the right table will be padded with nulls.

Syntax:

```
SELECT *  FROM TABLE1
LEFT JOIN TABLE2
ON TABLE1.COLUMN_NAME = TABLE2.COLUMN_NAME;
```

RIGHT (OUTER) JOIN:
All matching records from the left and right table are considered.
And all non-matching records in right table which does not match the left table will be padded with the with nulls.

Syntax:

```
SELECT *  FROM TABLE1
RIGHT JOIN TABLE2
ON TABLE1.COLUMN_NAME = TABLE2.COLUMN_NAME;
```

FULL (OUTER) JOIN:
All matching and non-matching records from both left and right join are considered.

Syntax:

```
SELECT *  FROM TABLE1
FULL OUTER JOIN TABLE2
ON TABLE1.COLUMN_NAME = TABLE2.COLUMN_NAME;
```

22. What is a Self-Join?

A table is joined to itself based on some relation between its own columns.

23. What is a Cross-Join?

Cross join is a Cartesian product where number of rows in the first table multiplied by number of rows in the second table.

If a WHERE clause is used in cross join, then the query will work like an INNER JOIN.

24. Natural Join (EQUI JOIN) in SQL?

joins two tables based on the same column name and datatypes. The resulting table will contain all the columns of both tables but keep only one common column.

Syntax:

SELECT * FROM table1 NATURAL JOIN table2;

25. Is A LEFT JOIN B the same as B RIGHT JOIN A?

Yes, both produce the same result, but the columns' order will reflect the order of the tables as specified in the query.

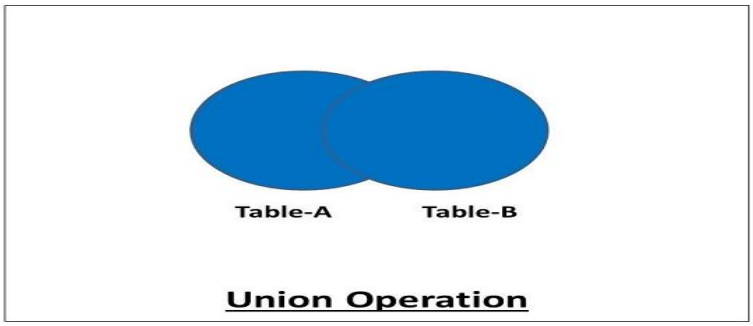
26. What is the difference between JOIN and UNION?

JOIN: Used to combine rows from multiple tables based on a related column between them. The resulting table will have more columns.

UNION: Used to combine two or more SELECT query results into single result. The resulting table will have more distinct rows.

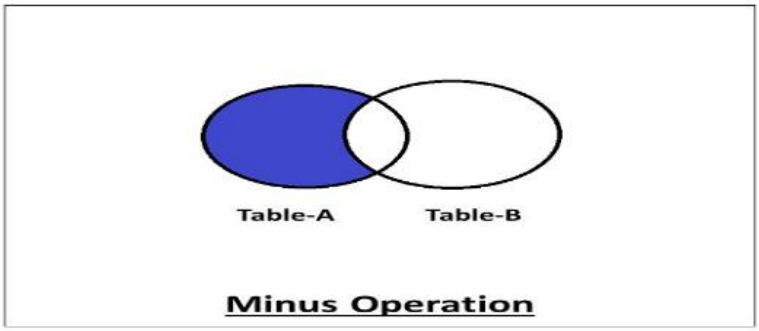
27. What is UNION, MINUS and INTERSECT commands?

1. **UNION:**
   - Used to combine the two or more select query results into single result.
   - It automatically removes duplicate rows from the results.

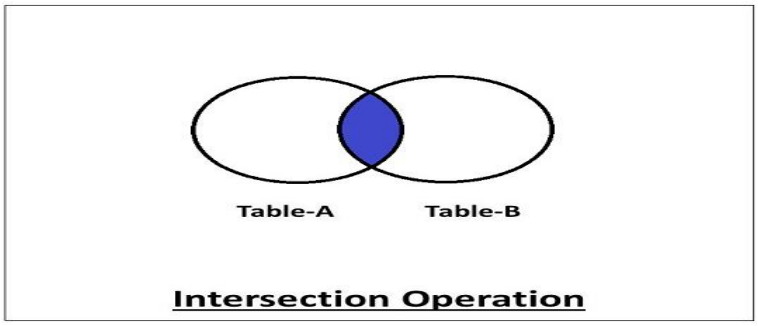


2. **UNION ALL:**
   - Used to combine the two or more select query results into single result.
   - UNION ALL will include duplicate records.

3. **MINUS:** returns the rows that are present only in the first select statement but not in the second select statement.



4. **INTERSECT:** returns only rows which are present in first select statement and second select statement.



28. Difference between Where and Having Clause?

Having: -

- HAVING Clause is used to filter records from the group by.
- HAVING Clause is used after GROUP BY Clause.
- HAVING Clause cannot be used without GROUP BY Clause
- HAVING Clause can only be used with SELECT statement.
- HAVING Clause can contain aggregations like SUM, AVG, MIN and MAX, COUNT etc.

Where: -

- WHERE Clause is used to filter the records from the table.
- WHERE Clause is used before GROUP BY Clause.
- WHERE Clause can be used with SELECT, UPDATE, DELETE statement.
- WHERE Clause is used with single row function like UPPER, LOWER etc.

We can use where and having in same query. Where is more performant than having.

29. What is the difference between order by and group by?

ORDER BY

- The ORDER BY clause is used to sort the data in ascending or descending order.
- ASC denotes ascending order, while DESC denotes descending order.

- ORDER BY guarantees global ordering.

GROUP BY

- The GROUP BY clause is used to organize data that have the same column values.
- Group by clause in collaboration with aggregate functions like SUM, AVG, MIN, MAX, and COUNT.

30. Differentiate between Rank, Row Number and Dense Rank?

Also known as window functions. These are used to give numbering based on one or more condition.

If there are no duplicates then row number, rank, dense rank leads to similar results. The only difference comes if there are duplicates.

row number:

- It allows Numbering to duplicate records.
- When we use row number, we should use order by clause.
- We can also use partition by clause is optional –then row number starts from 1 for each partition.

rank:

- for duplicates same rank is assigned and for the next entry it skips the next rank. (There are holes between ranks).
- Order by clause is mandatory.
- partition by clause is optional.

dense rank:

- for duplicates same rank assigned and next entry starts from next rank (there are no holes between ranks).
- Order by clause is mandatory.
- partition by clause is optional.

Scenario: whenever you don't have duplicates use row number.

31. What is a Slowly Changing Dimension?

A Slowly Changing Dimension (SCD) is a dimension that stores and manages both current and historical data over time in a data warehouse. It is considered and implemented as one of the most critical ETL tasks in tracking the history of dimension records.

- **SCD Type 0:** No changes; the dimensions are static.

- **SCD Type 1:**
  SCD type 1 methodology is used when there is no need to store historical data in the dimension table. This method overwrites the old data in the dimension table with the new data. It is used to correct data errors in the dimension. The advantage of type1 is ease of maintenance and less space occupied. The disadvantage is that there is no historical data kept in the data warehouse.

  - **Example,** i have the customer table with the below data.

surrogate_key	customer_id	customer_name	Location
1	101	Marspton	Illions

  - Here the customer's name is misspelt. It should be Marston instead of Marspton. If you use type1 method, it just simply overwrites the data. The data in the updated table will be.

surrogate_key	customer_id	customer_name	Location
1	101	Marston	Illions

- **SCD Type 2**:
  SCD type 2 stores the entire history the data in the dimension table. new and historical results captured in separate rows.With type 2 we can store unlimited history in the dimension table. In type 2, you can store the data in three different ways. They are
  - Versioning
  - Flagging
  - Effective Date and End date

- **SCD Type 2 Versioning**:
  In versioning method, a sequence number is used to represent the change. The latest sequence number always represents the current row and the previous sequence numbers represents the past data.
  - As an example, let's use the same example of customer who changes the location. Initially the customer is in Illions location and the data in dimension table will look as.

surrogate_key	customer_id	customer_name	Location	Version
1	101	Marston	Illions	1

- The customer moves from Illions to Seattle and the version number will be incremented. The dimension table will look as

surrogate_key	customer_id	customer_name	Location	Version
1	101	Marston	Illions	1
2	101	Marston	Seattle	2

- Now again if the customer is moved to another location, a new record will be inserted into the dimension table with the next version number.

- **SCD Type 2 Flagging**:
  In flagging method, a flag column is created in the dimension table. The current record will have the flag value as 1 and the previous records will have the flag as 0.

  - Now for the first time, the customer dimension will look as.

surrogate_key	customer_id	customer_name	Location	flag
1	101	Marston	Illions	1

  - Now when the customer moves to a new location, the old records will be updated with flag value as 0 and the latest record will have the flag value as

surrogate_key	customer_id	customer_name	Location	flag
1	101	Marston	Illions	0
2	101	Marston	Seattle	1

- **SCD Type 2 Effective Date and End date**:
  In Effective Date method, the period of the change is tracked using the start_date and end_date columns in the dimension table.
  - customer dimension will look as.

surrogate_key	customer_id	customer_name	Location	Start_date	End_date
1	101	Marston	Illions	01-Mar-2010	20-Fdb-2011
2	101	Marston	Seattle	21-Feb-2011	NULL

- The NULL in the End_Date indicates the current version of the data and the remaining records indicate the past data. Top of Form Bottom of Form

- **SCD Type 3**:
  In type 3 method, only the current status and previous status of the row is maintained in the table. To track these changes two separate columns are created in the table. The type 3 method will have limited history, and it depends on the number of columns you create.

  - **Example,** i have the customer table with the below data.

surrogate_key	customer_id	customer_name	Current_Location	previous_location
1	101	Marston	Illions	NULL

  - Let say, the customer moves from Illions to Seattle and the updated table will look as

surrogate_key	customer_id	customer_name	Current_Location	previous_location
1	101	Marston	Seattle	Illions

  - Now again if the customer moves from seattle to NewYork, then the updated table will be

surrogate_key	customer_id	customer_name	Current_Location	previous_location
1	101	Marston	NewYork	Seattle

- **SCD Type 4:** The historical data is maintained in a separate table. The current table maintains a single row and all history is kept in a separate table.

- **SCD Type 6:** Hybrid (Type 1 + Type 2 + Type 3). The table captures the historical rows and the current role maintains the current value and the previous value in two separate rows.

32. What are Dimension tables and fact tables?

Fact Tables:

- A fact table/ reality table is a primary table in a dimensional model.
- A Fact Table contains Measurements/facts and foreign key to dimension table.
- A fact is a piece of information with a specific numerical value.
- Facts are accompanied by dimensions.
- There are less attributes/columns than dimension table.
- There are more records than dimension table.
- Fact table forms a vertical table.
- It is used for analysis purpose and decision making.
- Fact tables store quantitative data (e.g., sales amounts)

Dimension tables:

- A dimension table contains dimensions of a fact.
- There are more attributes/columns than fact table.
- There are less records than the fact table.
- While dimension table forms a horizontal table.
- While the dimension table has a primary key, the table has a concatenated key.
- dimension tables provide context (e.g., customer details).

Example: Sales fact table linked with Date and Product dimensions for analysis.

33. What are Entities and Relationships?

Entity: An entity can be a real-world object. Each entity has some associated properties that provide it with an identity.

EX: college database, students, professors, workers, departments, and projects can be referred to as entities.

Relationships: Relations or links between entities that have something to do with each other.
EX: The employee's table in a company's database can be associated with the salary table in the same database.

34. List the different types of relationships in SQL.

One-to-One – In this relationship where each record in one table is associated with the maximum of one record in the other table.

One-to-Many & Many-to-One - This is the most used relationship where a record in a table is associated with multiple records in the other table vice versa.
Many-to-Many - This is used in cases when multiple records on both sides are needed for defining a relationship.
Self-Referencing Relationships - This is used when a table needs to define a relationship with itself.

35. What is Alias in SQL?

- It is a temporary name assigned to the table or table column for the purpose of a particular SQL query.
- In addition, is technique to secure the real names of database fields.
- A table alias is also called a correlation name.
- An alias is represented explicitly by the AS keyword but in some cases, the same can be performed without it as well.
- Nevertheless, using the AS keyword is always good practice.

```
SELECT A.emp_name AS "Employee" /* Alias using AS keyword */
B.emp_name AS "Supervisor"
FROM employee A, employee B /* Alias without AS keyword */
WHERE A.emp_sup = B.emp_id;
```

36. What are wildcards in SQL?

- Wildcards are special characters used with the 'LIKE' operator to filter data based on patterns:
- '%' represents any sequence of characters, making it useful for broad searches.
- '_' represents a single character, allowing for precise pattern matching with variability.

37. What is Pattern Matching in SQL?

Pattern matching is checking whether a specific sequence of characters or data exist within a dataset.

The LIKE operator is used with the WHERE clause. It enables users to use _ to match a single character and % to match an arbitrary number of characters.

1. Search a student in your database with first name beginning with the letter K:
   ```
   SELECT *
   FROM students
   WHERE first_name LIKE 'K%'
   ```

2. all students whose first name does not begin with K.
   ```
   SELECT *
   FROM students
   WHERE first_name NOT LIKE 'K%'
   ```

3. Search for a student in the database where he has a K in his first name.
   ```
   SELECT *
   FROM students
   WHERE first_name LIKE '%K%'
   ```

4. This query fetches all students with letter K at the third position in their first name.
   ```
   SELECT *
   FROM students
   WHERE first_name LIKE '__K%'
   ```

5. Matches first names with three or more letters.
   ```
   SELECT  *
   FROM students
   WHERE first_name LIKE '___%'
   ```

6. Matches first names with exactly four characters.
   ```
   SELECT  *
   FROM students
   WHERE first_name LIKE '____'
   ```

38. Difference Between LIKE and = in SQL ?

- '=' is used for exact comparisons between values, typically leveraging indexes for quick lookups.
- 'LIKE' is used for pattern matching with wildcards (`%` and `_`), which allows flexible searches but can lead to performance hits due to potential full table scans.

39. Why is = Faster than LIKE in SQL?

'=' utilizes indexes effectively because it searches for exact matches, resulting in faster query execution.

'LIKE', especially with leading wildcards, often bypasses indexes and performs sequential scans, making it significantly slower for large datasets.

40. What are the differences between OLTP and OLAP?

OLTP:

- OLTP stands for Online Transaction Processing.
- These systems are generally designed for a large audience of end-users who conduct short transactions.
- Queries involved in such databases are generally simple, need fast response times, and return relatively few records.
- Several transactions per second act as an effective measure for such systems.
- Use Case: Designed for managing transactional data (e.g., banking systems, online shopping).
- Example: A retail transaction system where every sale, return, or inquiry is processed immediately.
- EX: oracle databases.

OLAP:

- OLAP stands for Online Analytical Processing.
- Queries are too complex and involve a bunch of aggregations.
- For OLAP systems, the effectiveness measure relies highly on response time.
- Such systems are widely used for data mining or maintaining aggregated, historical data, usually in multi-dimensional schemas.
- Use Case: Designed for data analysis and complex queries (e.g., business reporting, data mining).
- Example: A sales data analysis system that aggregates data across multiple dimensions (time, region, product).
- EX: data warehouse

41. What are Aggregate and Scalar functions?

Aggregate functions are used with the GROUP BY and HAVING clauses of the SELECT statement. Following are the widely used SQL aggregate functions:

- AVG () - Calculates the meaning of a collection of values.
- COUNT () - Counts the total number of records in a specific table or view.
- MIN () - Calculates the minimum of a collection of values.
- MAX () - Calculates the maximum of a collection of values.
- SUM () - Calculates the sum of a collection of values.
- FIRST () - Fetches the first element in a collection of values.
- LAST () - Fetches the last element in a collection of values.

A scalar function returns a single value based on the input value. Following are the widely used SQL scalar functions:

- LEN () - Calculates the total length of the given field (column).
- UPPER () - Converts a collection of string values to uppercase characters.
- LOWER () - Converts a collection of string values to lowercase characters.
- STR () - Extracts substrings from a collection of string values in a table.
- CONCAT () - Concatenates two or more strings.
- RAND () - Generates a random collection of numbers of a given length.
- ROUND () - Calculates the round-off integer value for a numeric field (or decimal point values).
- NOW () - Returns the current date & time.
- FORMAT () - Sets the format to display a collection of values.
- COALESCE ()

42. What is User-defined function? What are its various types?

The user-defined functions in SQL are like functions in any other programming language that accept parameters, perform complex calculations, and return a value. They are written to use the logic repetitively whenever required.

There are two types of SQL user-defined functions:

- **Scalar Function:** As explained earlier, user-defined scalar functions return a single scalar value.
- **Table-Valued Functions:** User-defined table-valued functions return a table as output.
- **Inline:** returns a table data type based on a single SELECT statement.
- **Multi-statement:** returns a tabular result-set but, unlike inline, multiple SELECT statements can be used inside the function body.

43. What is an Index? Explain its different types?

An index is a database object that improves query performance by allowing faster data retrieval. It works like a book index - helping you find data quickly without scanning the entire table.

Types of Indexes:

1. **Clustered Index:**
   PRIMARY KEY is the clustered index.
   The table's data is physically stored in the order of this index.
   There can be only one clustered index per table.

   Example:
   SQLCREATE TABLE customers (

```
customer_id INT PRIMARY KEY,   #Clustered index
name VARCHAR(100),
email VARCHAR(200),
city VARCHAR(200));
```

2. **Non-Clustered Index:**
   Creates a separate structure that points to the data without changing the physical order. All other indexes in MySQL (besides the PRIMARY KEY) are non-clustered. These indexes created on Non primary columns.

   Types of Non-Clustered Indexes:

   UNIQUE Index: Prevents duplicates.
   Example: CREATE UNIQUE INDEX idx_email ON customers(email);

   Secondary Index (Non-Unique): Speeds up filtering.
   Example: CREATE INDEX idx_city ON customers(city);

   Composite Index: Multi-column index (follows leftmost prefix rule).
   Example: CREATE INDEX idx_city_name ON customers (city, name);

44. What is Index Partitioning?

Index partitioning divides a large index into smaller partitions for better performance and manageability. It can be done in different ways, such as range, list, or hash partitioning.

```
CREATE INDEX idx_orders ON orders(order_date)
PARTITION BY RANGE (order_date) (
PARTITION p1 VALUES LESS THAN ('2022-01-01'),
PARTITION p2 VALUES LESS THAN ('2023-01-01'),
PARTITION p3 VALUES LESS THAN (MAXVALUE)
);
```

45. What is Data Integrity?

Data Integrity defines the accuracy and consistency of data stored in a database. It can also define integrity constraints to enforce business rules on the data when it is entered into the application or database.

46. What is a Subquery? What are its types?

A subquery is a query within another query, also known as a nested query or inner query. The outer query is called as main query, and inner query is called subquery. Subquery is always executed first, and the result of subquery is passed on to the main query.

There are two types of subqueries – Correlated and Non-Correlated.

- correlated subquery query is dependent query.
- non-Correlated sub query can be considered as independent query.

47. What are correlated and sub-queries in sql?

In a situation where inner query and outer query want to execute on a common group by condition. Then we use correlated sub-queries.

48. What is CTE in SQL?

A Common Table Expression is a temporary result set used to simplify complex queries and make them more readable.

49. What are subqueries and CTEs, and which is preferable?

- Subqueries are nested queries inside a main query, often used for filtering, calculations, or transforming data within the same SQL statement.
- CTEs are Defined using WITH clauses, CTEs are temporary result sets that can be referenced multiple times within a query, making the SQL easier to read and maintain.
- CTEs are often better for readability, debugging, and reusability, especially in complex queries.

50. What is Normalization? What are benefits and different types?

Normalization is a database design technique that organizes data into structured tables to reduce data redundancy (duplicate data) and prevent data anomalies (issues during insert, update, or delete). It achieves this by breaking large, unorganized tables into smaller, related tables using primary keys (unique identifiers) and foreign keys (links between tables).

Benefits:

- Removes redundancy → No repeated data
- Ensures data integrity → Accurate and consistent

- Simplifies maintenance → Easy updates and changes
- Improves performance → Faster queries in most cases

- **1NF (First Normal Form):** 1NF ensures that every column contains atomic values (indivisible) and there are no repeating groups or arrays in a table. Each row must be unique, and every column should store a single value of a specific data type.

  Example: Instead of storing multiple phone numbers in one column, create separate rows for each number.

- **2NF (Second Normal Form):** 2NF applies when a table is in 1NF and removes partial dependency, which means a non-key column should not depend on part of a composite primary key. Every non-key attribute must depend on the entire primary key.

  Example: If StudentName depends only on StudentID in a table with (StudentID, CourseID) as key, move StudentName to a separate table.

- **3NF (Third Normal Form):**3NF applies when a table is in 2NF and removes transitive dependency, meaning non-key columns should not depend on other non-key columns. Every non-key attribute must depend only on the primary key.

  Example: If City depends on ZipCode, and ZipCode depends on CustomerID, move City and ZipCode to a separate table.

- **BCNF (Boyce-Codd Normal Form):** Stricter than 3NF. Every determinant (column that decides another column) must be a candidate key.

  Example: If Teacher decides Subject but Teacher is not a key, split the table.

- **4NF (Fourth Normal Form):** Removes multi-valued dependencies—no column should have multiple independent sets of values.

  Example: A person with multiple phone numbers and multiple emails → separate into different tables.

- **5NF (Fifth Normal Form):** Removes join dependencies—break tables so no unnecessary recombination is needed.

  Example: Complex relationships split into smallest possible tables.

51. **What is Denormalization?**

Denormalization is the opposite of normalization; it's a database optimization technique where redundant data is intentionally added or tables are combined to improve **read performance** and reduce complex joins.

Benefits: Faster queries, better reporting, and simpler data retrieval.

52. **ACID properties**?

Acid properties ensure that database transactions are **reliable**, even in cases of system failures, concurrency issues, or unexpected errors.
ACID stands for:

- Atomicity:
- Consistency:
- Isolation:
- Durability:

1. **Atomicity:**
   A transaction is treated as a single unit.Either all operations succeed, or none are applied.

   Example: Transferring money between accounts—debit and credit must both happen, or neither.

2. **Consistency:**
   The database moves from one valid state to another.All integrity constraints (like foreign keys, unique constraints) remain satisfied.

   Example: After a transaction, account balances should still obey business rules.

3. **Isolation:**
   Concurrent transactions do not interfere with each other.Intermediate states of a transaction are not visible to others.

   Example: Two users booking the last seat should not both succeed.

4. **Durability:**
   Once a transaction is committed, changes are permanent—even after a crash.
   Achieved via write-ahead logs or persistent storage.

53. What is a Stored Procedure?

A stored procedure is a precompiled collection of SQL statements stored in the database, which can be executed as a single unit.

54. What are the advantages of using Stored Procedures?

It improves performance, reusability, and security, Maintainability.

- Performance: Precompiled, so faster execution.
- Reusability: Can be reused across applications.
- Security: Can restrict direct table access.
- Maintainability: Centralized logic in the database.

55. Query Optimization: How would you optimize a slow-running query in a large-scale relational database?

To optimize SQL queries:

- Use indexes to speed up search operations.
- Select only the necessary columns instead of using 'SELECT *'.
- Simplify joins and avoid unnecessary subqueries.
- Analyze execution plans to identify slow operations.

56. What is a Pivot in SQL?

A pivot converts row-based data into columns, enabling better summaries and data aggregation for reporting.

57. What is Schema Evolution?

Schema evolution allows changes to the schema without rewriting data, helpful in handling growing datasets or new attributes.

58. Explain Apply Operator in SQL?

CROSS APPLY acts like an INNER JOIN for table-valued functions, while OUTER APPLY is similar to a LEFT JOIN, including unmatched rows.